

Know Your Tools: Version Control and Build Systems

Martin Kellogg

Know Your Tools: VCS and Build Systems

Today's agenda:

- **Reading Quiz**
- Version control:
 - How does a version control system work?
 - How to use your VCS
 - GitHub workflows
- Build systems:
 - What is a build system? How does one work?
 - How to choose a build system + best practices

Announcements

- You should have already had your kickoff meeting with your group project mentor
 - If you haven't had it yet, you're behind
 - If you haven't scheduled it yet, you're losing participation points

Announcements

- You should have already had your kickoff meeting with your group project mentor
 - If you haven't had it yet, you're behind
 - If you haven't scheduled it yet, you're losing participation points
- Deployment demo must happen during your 2nd meeting with your team's mentor
 - due "this week", but hard deadline is next Monday, not Friday

Announcements

- You should have already had your kickoff meeting with your group project mentor
 - If you haven't had it yet, you're behind
 - If you haven't scheduled it yet, you're losing participation points
- Deployment demo must happen during your 2nd meeting with your team's mentor
 - due "this week", but hard deadline is next Monday, not Friday
- Group project proposals due next Wednesday (10/7)

Announcements

- You should have already had your kickoff meeting with your group project mentor
 - If you haven't had it yet, you're behind
 - If you haven't scheduled it yet, you're losing participation points
- Deployment demo must happen during your 2nd meeting with your team's mentor
 - due "this week", but hard deadline is next Monday, not Friday
- Group project proposals due next Wednesday (10/7)
- First midterm exam coming up (10/12)

Announcements

- You should have already had your kickoff meeting with your group project mentor
 - If you haven't had it yet, you're behind
 - If you haven't scheduled it yet, you're losing participation points
- Deployment demo must happen during your 2nd meeting with your team's mentor
 - due "this week", but hard deadline is next Monday, not Friday
- Group project proposals due next Wednesday (10/7)
- First midterm exam coming up (10/12)
- I am out of town 10/5 to 10/16; Jeri Qerimaj will lecture

Know Your Tools: VCS and Build Systems

Today's agenda:

- Reading Quiz
- **Version control:**
 - How does a version control system work?
 - How to use your VCS
 - GitHub workflows
- **Build systems:**
 - What is a build system? How does one work?
 - How to choose a build system + best practices

Let's share a file

Let's share a file



Let's share a file



These systems are fine for “**binary blobs**”: files that you don't intend to change once shared

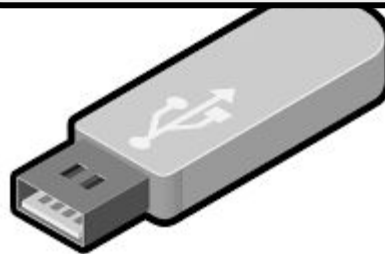


Let's share a file



These systems are fine for “**binary blobs**”: files that you don't intend to change once shared

- **but not for code**



What is “version control”?

Definition: a *version control system* is a program that manages many versions of one or more text-based documents by storing diffs between them. Its goals are:

What is “version control”?

Definition: a *version control system* is a program that manages many versions of one or more text-based documents by storing diffs between them. Its goals are:

- Keep a *history* of your work
 - Explain the purpose of each change
 - Checkpoint specific versions (known good state)
 - Recover specific state (fix bugs, test old versions)

What is “version control”?

Definition: a *version control system* is a program that manages many versions of one or more text-based documents by storing diffs between them. Its goals are:

- Keep a *history* of your work
 - Explain the purpose of each change
 - Checkpoint specific versions (known good state)
 - Recover specific state (fix bugs, test old versions)
- *Coordinate*/merge work between team members
 - Or yourself, on multiple computers, or multiple features

Coordinating with others

- `pull` incorporates others' changes into your repository
 - (update brings changes into your working copy)
 - (N.b.: `git pull` does pull, merge, and update)

Coordinating with others

- `pull` incorporates others' changes into your repository
 - (update brings changes into your working copy)
 - (N.b.: `git pull` does pull, merge, and update)
- If you are **behind**, nothing more to do
 - Behind = your history is a **prefix** of master history

Coordinating with others

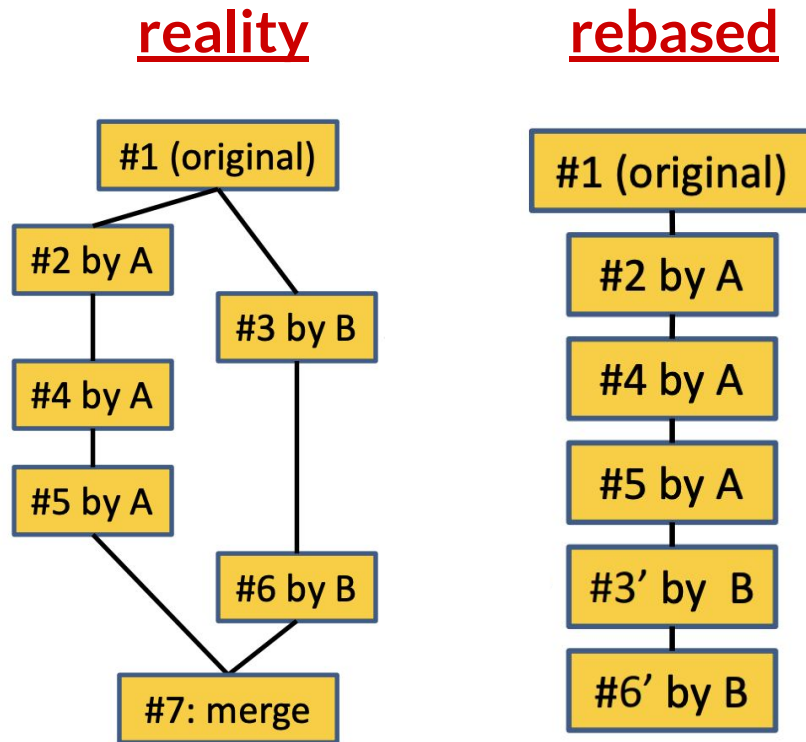
- `pull` incorporates others' changes into your repository
 - (update brings changes into your working copy)
 - (N.b.: `git pull` does pull, merge, and update)
- If you are **behind**, nothing more to do
 - Behind = your history is a **prefix** of master history
- If you have made changes in parallel, you must **merge**
 - Merge = create a new version incorporating all changes

Coordinating with others: rebasing

- rebase **rewrites** history

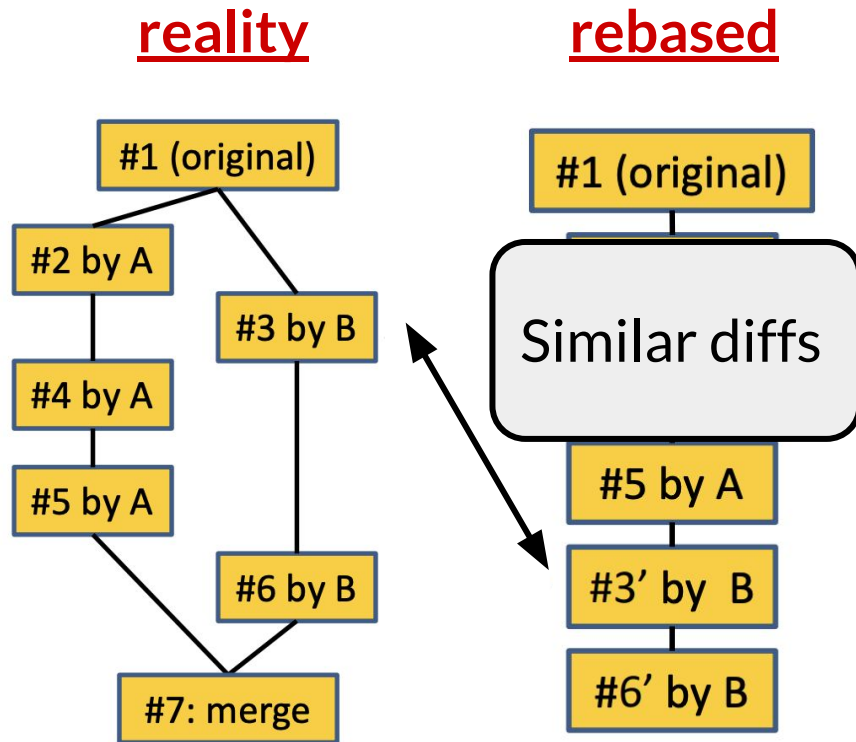
Coordinating with others: rebasing

- rebase **rewrites** history



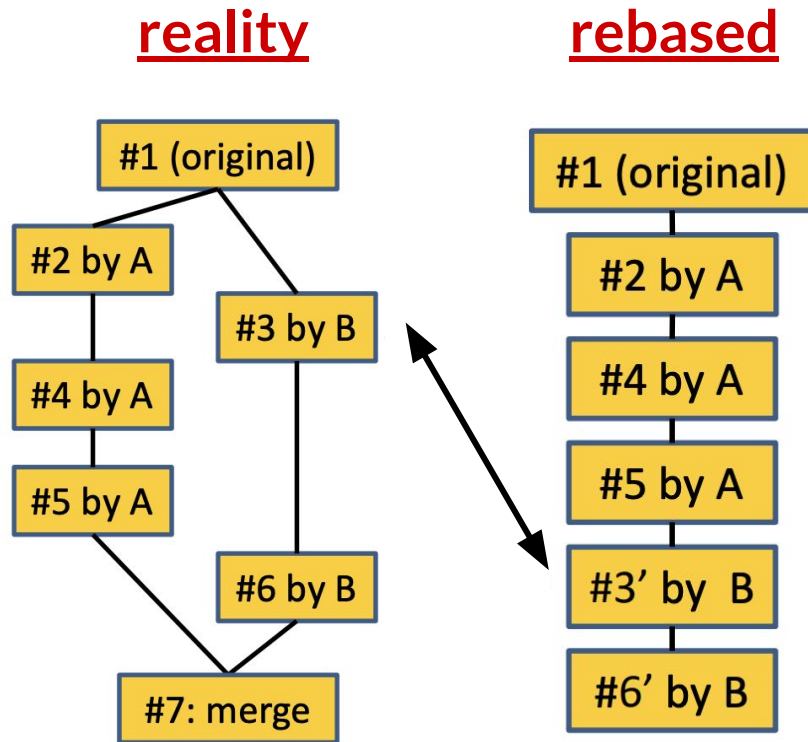
Coordinating with others: rebasing

- rebase **rewrites** history



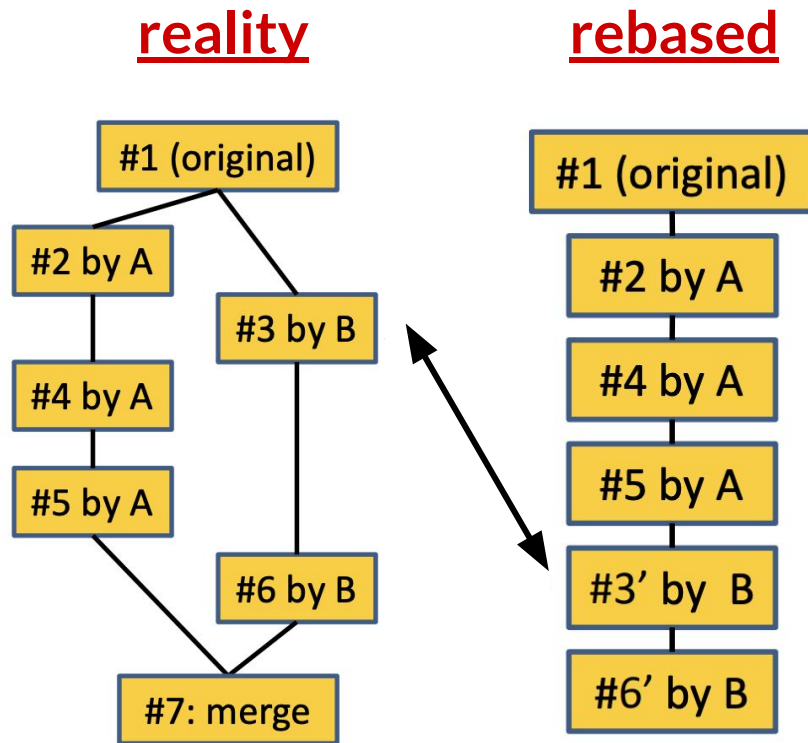
Coordinating with others: rebasing

- rebase **rewrites** history
- **Cleaner** history, easier to read



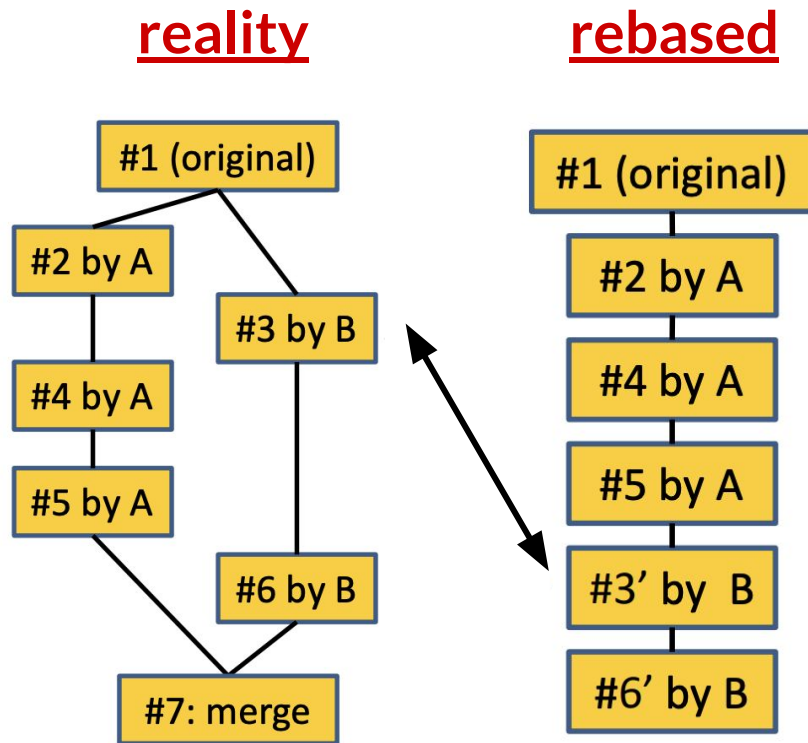
Coordinating with others: rebasing

- rebase **rewrites** history
- **Cleaner** history, easier to read
- Mixes commits #3 and #7
- Does not show context for change #3



Coordinating with others: rebasing

- rebase **rewrites** history
- **Cleaner** history, easier to read
- Mixes commits #3 and #7
- Does not show context for change #3
- Squash-and-merge is a safer form of rebasing



Coordinating with others: conflicts

Two changes can either be:

- **Conflict-free:**
- **Conflicting:**

Coordinating with others: conflicts

Two changes can either be:

- **Conflict-free**: changes are to different lines of a file
- **Conflicting**:

Coordinating with others: conflicts

Two changes can either be:

- **Conflict-free**: changes are to different lines of a file
- **Conflicting**:
 - Simultaneous changes to the same lines of a file
 - Requires manual conflict resolution

Coordinating with others: conflicts

Two changes can either be:

- **Conflict-free**: changes are to different lines of a file
- **Conflicting**:
 - Simultaneous changes to the same lines of a file
 - Requires manual conflict resolution

“Conflict-free” is a **textual, not semantic**, notion

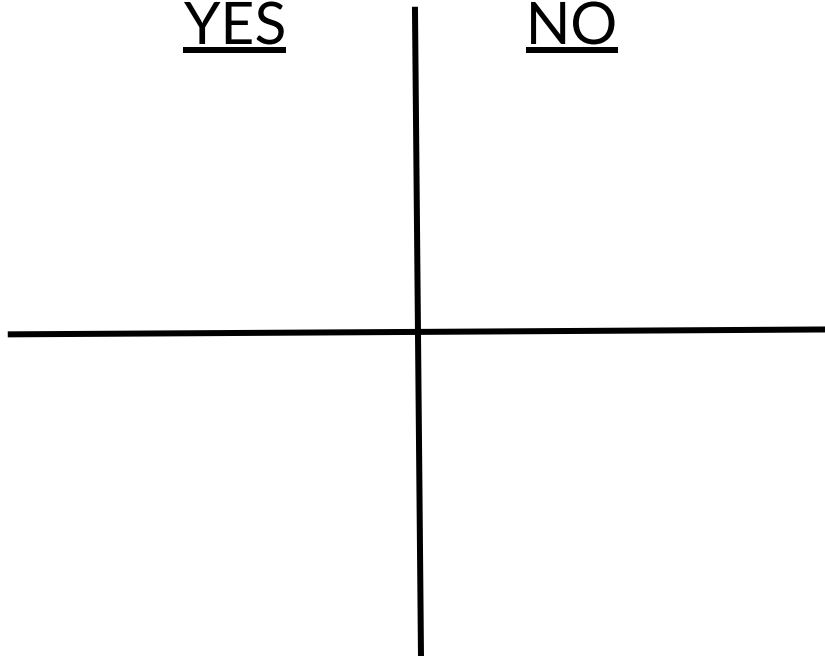
- A heuristic about when to get the user involved
- Could yield compile errors or test failures

Aside: false positives and false negatives

Can X **actually** happen?

YES

NO



Aside: false positives and false negatives

Can X actually happen?		
<u>YES</u>	<u>NO</u>	
<u>YES</u>		
<u>NO</u>		

Aside: false positives and false negatives

Can X actually happen?			
		<u>YES</u>	<u>NO</u>
Did a tool warn us about X?	<u>YES</u>	True positive	
	<u>NO</u>		

Aside: false positives and false negatives

Can X actually happen?	
<u>YES</u>	<u>NO</u>
<u>YES</u>	True positive
<u>NO</u>	False positive

Aside: false positives and false negatives

Can X actually happen?			
		<u>YES</u>	<u>NO</u>
Did a tool warn us about X?	<u>YES</u>	True positive	False positive
	<u>NO</u>	False negative	

Aside: false positives and false negatives

Can X actually happen?			
		<u>YES</u>	<u>NO</u>
Did a tool warn us about X?	<u>YES</u>	True positive	False positive
	<u>NO</u>	False negative	True negative

Aside: false positives and false negatives

Can X actually happen?		
<u>YES</u>	<u>NO</u>	
<u>YES</u>	True positive	Useful tool for thinking about anything that might warn us about a problem
<u>NO</u>	False negative	

Coordinating with others: conflicts

Two changes can either be:

- **Conflict-free**: changes are to different lines of a file
- **Conflicting**:
 - Simultaneous changes to the same
 - Requires manual conflict resolution

False positives,
false negatives,
both, or neither?

“Conflict-free” is a **textual**, **not semantic**, notion

- A heuristic about when to get the user involved
- Could yield compile errors or test failures

Coordinating with others: conflicts

Two changes can either be:

- **Conflict-free**: changes are to different lines of a file
- **Conflicting**:
 - Simultaneous changes to the same
 - Requires manual conflict resolution

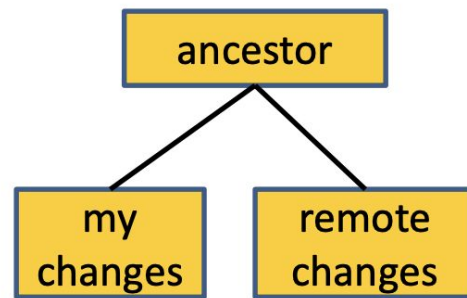
False positives,
false negatives,
both, or neither?

“Conflict-free” is a **textual**, **not semantic**, notion

- A **heuristic** about when to get the user involved
- Could yield compile errors or test failures

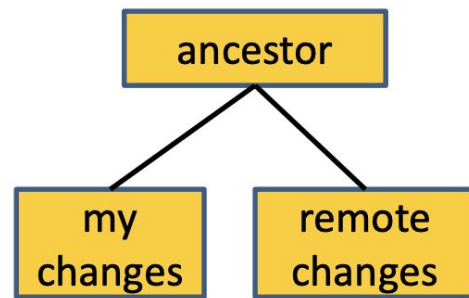
Coordinating with others: resolving conflicts

- There are **three versions** of the file:
- **You decide** which version to keep or how to merge them



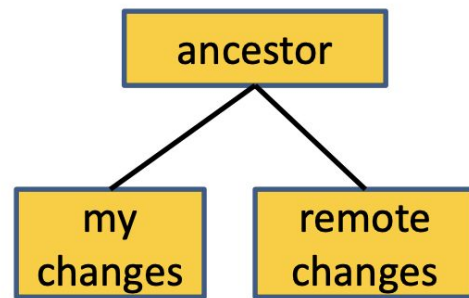
Coordinating with others: resolving conflicts

- There are **three versions** of the file:
- **You decide** which version to keep or how to merge them
- Many merge tools exist
- Configure your VCS to use the merge tool that you prefer
 - **Practice** this ahead of time!



Coordinating with others: resolving conflicts

- There are **three versions** of the file:
- **You decide** which version to keep or how to merge them
- Many merge tools exist
- Configure your VCS to use the merge tool that you prefer
 - **Practice** this ahead of time!
- **Don't panic!** Instead, think.
- **You can always** bail out of the merge and **start over**
 - You have the full local and remote history



Know Your Tools: VCS and Build Systems

Today's agenda:

- Reading Quiz
- Version control:
 - How does a version control system work?
 - **How to use your VCS**
 - GitHub workflows
- Build systems:
 - What is a build system? How does one work?
 - How to choose a build system + best practices

Version Control: advice and best practices

Best practice: don't commit binary files

- The history database records **changes**, not the entire file every time you commit

Best practice: don't commit binary files

- The history database records **changes**, not the entire file every time you commit
- **Avoid** binary files for content (especially simultaneous editing)
 - Word .docx files, Excel .xlsx files, other **proprietary formats**

Best practice: don't commit binary files

- The history database records **changes**, not the entire file every time you commit
- **Avoid** binary files for content (especially simultaneous editing)
 - Word .docx files, Excel .xlsx files, other **proprietary formats**
- Do not commit **generated files**, such as:
 - Binaries (e.g., .class files), etc.
 - IDE files (your teammates might use other tooling)

Best practice: don't commit binary files

- The history database records **changes**, not the entire file every time you commit
- **Avoid** binary files for content (especially simultaneous editing)
 - Word .docx files, Excel .xlsx files, other **proprietary formats**
- Do not commit **generated files**, such as:
 - Binaries (e.g., .class files), etc.
 - IDE files (your teammates might use other tooling)
 - Wastes space in repository
 - Causes merge conflicts

Best practice: feature branch development

- Whenever you start working on something new, create a branch
 - colloquially called a *feature branch*, even when it's not a feature

Best practice: feature branch development

- Whenever you start working on something new, create a branch
 - colloquially called a *feature branch*, even when it's not a feature
- Pros:
 - features developed in isolation (less risk of main being broken)
 - encourages small PRs
- Cons:
 - large features can make integration difficult

Best practice: feature branch development

- Whenever you start working on something new, create a branch
 - colloquially called a *feature branch* feature
- Pros:
 - features developed in isolation (leaves main branch stable)
 - encourages small PRs
- Cons:
 - large features can make integration difficult

Advice: use feature branch development model iff your team typically ships features quickly

Advice: synchronize with teammates often

- Pull often

Advice: synchronize with teammates often

- Pull often
 - **Avoid getting behind** the main repo or your teammates
 - Avoid difficult and/or complex merges

Advice: synchronize with teammates often

- Pull often
 - **Avoid getting behind** the main repo or your teammates
 - Avoid difficult and/or complex merges
- Push as often as practical

Advice: synchronize with teammates often

- Pull often
 - **Avoid getting behind** the main repo or your teammates
 - Avoid difficult and/or complex merges
- Push as often as practical
 - Don't let your teammates get behind you!
 - Don't destabilize the main build
 - Avoid long periods working on a branch
 - but do work in a feature branch - don't work directly on main!

Advice: commit messages

- Always write a commit message **yourself**

Advice: commit messages

- Always write a commit message **yourself**
 - **never** use an auto-generated message from a tool like “update *filename(s)*” from GitHub’s GUI

Advice: commit messages

- Always write a commit message **yourself**
 - **never** use an auto-generated message from a tool like “update *filename(s)*” from GitHub’s GUI
- Commit messages should be **descriptive**

Advice: commit messages

- Always write a commit message **yourself**
 - **never** use an auto-generated message from a tool like “update *filename(s)*” from GitHub’s GUI
- Commit messages should be **descriptive**
- Don’t write a novel: **summarize**. The code documentation in the commit should cover the rest.

Advice: commit messages: good or bad?

```
commit 763fe9cc335bb78ca45a608fa1f4c606713d5b44
```

```
Author:
```

```
Date:
```

```
Simplify `getImmediateSubcheckerClasses()` implementation (#5579)
```

Advice: commit messages: good or bad?

```
commit 763fe9cc335bb78ca45a608fa1f4c606713d5b44
```

```
Author:
```

```
Date:
```

```
Simplify `getImmediateSubcheckerClasses()` implementation (#5579)
```

GOOD: short and to the point. Contains link to the PR it was merged in

Advice: commit messages: good or bad?

```
commit 123317b24a72215071a0f02e08635ee4b5b9669a
```

```
Author: [REDACTED] <[REDACTED]@noreply.github.com>
```

```
Date: [REDACTED]
```

```
Update the code (#5)
```

Advice: commit messages: good or bad?

```
commit 123317b24a72215071a0f02e08635ee4b5b9669a
```

```
Author: [REDACTED] <[REDACTED]@noreply.github.com>
```

```
Date: [REDACTED]
```

```
Update the code (#5)
```

NOT SO GOOD:

description is vague
(looks auto-generated!)

Advice: commit messages: good or bad?

```
commit ddb6ab4df36a6bac3d4b118d40278f3428029f0c
```

```
Author: [REDACTED]@virginia.edu>
```

```
Date: [REDACTED] 2014 -0500
```

```
Comments? My code is self documenting.
```

Advice: commit messages: good or bad?

```
commit ddb6ab4df36a6bac3d4b118d40278f3428029f0c
```

```
Author: [REDACTED]@virginia.edu>
```

```
Date: [REDACTED] 2014 -0500
```

```
Comments? My code is self documenting.
```

NOT SO GOOD: while
the humor is nice, this
message is content-free

Advice: commit early and often

Advice: commit early and often

- Make **many small commits**, not one big one

Advice: commit early and often

- Make **many small commits**, not one big one
 - **Easier** to understand, review, merge, revert

Advice: commit early and often

- Make **many small commits**, not one big one
 - **Easier** to understand, review, merge, revert
- How to make many small commits:

Advice: commit early and often

- Make **many small commits**, not one big one
 - **Easier** to understand, review, merge, revert
- How to make many small commits:
 - Do only **one task at a time** and commit after each one

Advice: commit early and often

- Make **many small commits**, not one big one
 - **Easier** to understand, review, merge, revert
- How to make many small commits:
 - Do only **one task at a time** and commit after each one
 - Do multiple tasks in one working copy
 - Commit only a subset of files (use git's staging area)
 - Error-prone

Advice: commit early and often

- Make **many small commits**, not one big one
 - **Easier** to understand, review, merge, revert
- How to make many small commits:
 - Do only **one task at a time** and commit after each one
 - Do multiple tasks in one working copy
 - Commit only a subset of files (use git's staging area)
 - Error-prone
 - Create a branch for each simultaneous task
 - Need to keep track of all your branches, merge
 - Easier to share unfinished work with teammates

Advice: ways to avoid merge conflicts

- **Modularize** your work
 - Divide work so that individuals or subteams “**own**” a module
 - Other team members only need to understand its specification (abstractions!)
 - Requires good documentation and testing

Advice: ways to avoid merge conflicts

- **Modularize** your work
 - Divide work so that individuals or subteams “**own**” a module
 - Other team members only need to understand its specification (abstractions!)
 - Requires good documentation and testing

Bonus: this kind of modularization improves **observability** for management: it's easier to see who is being productive

Advice: ways to avoid merge conflicts

- **Modularize** your work
 - Divide work so that individuals or subteams “**own**” a module
 - Other team members only need to understand its specification (abstractions!)
 - Requires good documentation and testing
- **Communicate** about changes that may conflict
 - Don't overwhelm the team with such messages

Advice: always use version control

Advice: always use version control

- Still worthwhile, **even when working alone**
 - backups
 - feature branches are still useful when working on multiple parts of a system in parallel
 - sharing work across multiple computers

Advice: always use version control

- Still worthwhile, **even when working alone**
 - backups
 - feature branches are still useful when working on multiple parts of a system in parallel
 - sharing work across multiple computers
- Use **private repos** for things that should be private
 - GitHub will give you free private repos because you're students

Advice: always use version control

- Still worthwhile, **even when working alone**
 - backups
 - feature branches are still useful when working on multiple parts of a system in parallel
 - sharing work across multiple
- Use **private repos** for things that
 - GitHub will give you free private repos for students

I use **text-based formats** for many files so that I can version control them

Know Your Tools: VCS and Build Systems

Today's agenda:

- Reading Quiz
- Version control:
 - How does a version control system work?
 - How to use your VCS
 - **GitHub workflows**
- Build systems:
 - What is a build system? How does one work?
 - How to choose a build system + best practices

How to make a PR on GitHub

- start by creating a *fork* of the project
 - a new repository controlled by you, connected to the main

How to make a PR on GitHub

- start by creating a **fork** of the project
 - a new repository controlled by you, connected to the main
- in your fork, create a **feature branch**

How to make a PR on GitHub

- start by creating a **fork** of the project
 - a new repository controlled by you, connected to the main
- in your fork, create a **feature branch**
- write code + tests

How to make a PR on GitHub

- start by creating a **fork** of the project
 - a new repository controlled by you, connected to the main
- in your fork, create a **feature branch**
- write code + tests
- commit **early and often**, push to your fork

How to make a PR on GitHub

- start by creating a **fork** of the project
 - a new repository controlled by you, connected to the main
- in your fork, create a **feature branch**
- write code + tests
- commit **early and often**, push to your fork
- **prepare** for code review: follow code review author's best practices

How to make a PR on GitHub

- start by creating a **fork** of the project
 - a new repository controlled by you, connected to the main
- in your fork, create a **feature branch**
- write code + tests
- commit **early and often**, push to your fork
- **prepare** for code review: follow code review author's best practices
 - we'll discuss how to do a code review next lecture

How to make a PR on GitHub

- start by creating a **fork** of the project
 - a new repository controlled by you, connected to the main
- in your fork, create a **feature branch**
- write code + tests
- commit **early and often**, push to your fork
- **prepare** for code review: follow code review author's best practices
 - we'll discuss how to do a code review next lecture
- open PR against "**main**" repository from your fork's feature branch

How NOT to make a PR on GitHub

- start by creating a *hard fork* of the project
 - a new repository controlled by you, unconnected to the main

How NOT to make a PR on GitHub

- start by creating a *hard fork* of the project
 - a new repository controlled by you, unconnected to the main
- do all of your work on the repository's **main** branch

How NOT to make a PR on GitHub

- start by creating a *hard fork* of the project
 - a new repository controlled by you, unconnected to the main
- do all of your work on the repository's *main* branch
- write code (if there are already tests, don't bother to run them)

How NOT to make a PR on GitHub

- start by creating a *hard fork* of the project
 - a new repository controlled by you, unconnected to the main
- do all of your work on the repository's **main** branch
- write code (if there are already tests, don't bother to run them)
- commit **all** of your code at once, when you're done

How NOT to make a PR on GitHub

- start by creating a **hard fork** of the project
 - a new repository controlled by you, unconnected to the main
- do all of your work on the repository's **main** branch
- write code (if there are already tests, don't bother to run them)
- commit **all** of your code at once, when you're done
- **don't bother** to check if you've followed best practices

How NOT to make a PR on GitHub

- start by creating a **hard fork** of the project
 - a new repository controlled by you, unconnected to the main
- do all of your work on the repository's **main** branch
- write code (if there are already tests, don't bother to run them)
- commit **all** of your code at once, when you're done
- **don't bother** to check if you've followed best practices
- **email** your changes to the maintainer of the original project

How NOT to make a PR on GitHub

- start by creating a **hard fork** of the project
 - a new repository controlled by you, unconnected to the main
- do all of your work on the repository's **main** branch
- write code (if there are already tests, don't bother to run them)
- commit **all** of your code at once, when you're done
- **don't bother** to check if you've followed best practices
- **email** your changes to the maintainer of the original project
 - bonus points: email the full working copy, not just the diffs

How NOT to make a PR on GitHub

- start by creating a **hard fork** of the project
 - a new repository controlled by you, unconnected to the main
- do all of your work on the repository's **main** branch
- write code (if there are already tests, don't bother to run them)
- commit **all** of your code at once, when you're done
- **don't bother** to check if you've followed the project's guidelines
- **email** your changes to the maintainers
 - bonus points: email the full working directory

I've seen people make
all of these mistakes
(and more)!

Know Your Tools: VCS and Build Systems

Today's agenda:

- Reading Quiz
- Version control:
 - How does a version control system work?
 - How to use your VCS
 - GitHub workflows
- **Build systems:**
 - What is a build system? How does one work?
 - How to choose a build system + best practices

From the reading

“Here's how most clients I work with build a project:

1. Open the IDE
2. Load the solution
3. Get latest
4. Press F5 (or CTRL+SHIFT+B)”

From the reading

“Here's how most clients I work with build a new project:

1. Open the IDE
2. Load the solution
3. Get latest
4. Press F5 (or CTRL+SHIFT+F5)

“The F5 key is not a build process. It's a quick and dirty substitute. If that's how you build your software, I regret that I have to be the one to tell you this, but *your project is not based on solid software engineering practices.*”

From the reading

“Here's how most clients I work with build a project:

1. Open the IDE
2. Load the solution
3. Get latest
4. Press F5 (or CTRL+SHIFT+F5)

“The F5 key is not a build process. It's a quick and dirty substitute. If that's how you build your software, I regret that I have to be the one to tell you this, but *your project is not based on solid software engineering practices.*”

Key objective of a build system: avoid this problem!

What is a build system?

What is a build system?

Definition: A *build system* is a tool for orchestrating software engineering tasks

What is a build system?

Definition: A *build system* is a tool for orchestrating software engineering tasks

- Getting the source code
- Installing dependencies
- Compiling the code
- Running static analysis
- Generating documentation
- Running tests
- Creating artifacts for customers
- Shipping!

What is a build system?

Definition: A *build system* is a tool for orchestrating software engineering tasks

- Getting the source code
- Installing dependencies
- Compiling the code
- Running static analysis
- Generating documentation
- Running tests
- Creating artifacts for customers
- Shipping!



**A good build system
handles all these**

Tasks

Definition: a *task* is anything that the build system can do

Tasks

Definition: a *task* is anything that the build system can do

- Getting the source code
- Installing dependencies
- Compiling the code
- Running static analysis
- Generating documentation
- Running tests
- Creating artifacts for customers
- Shipping!

Tasks

Definition: a *task* is anything that the build system can do

- Getting the source code
- Installing dependencies
- Compiling the code
- Running static analysis
- Generating documentation
- Running tests
- Creating artifacts for customers
- Shipping!



All tasks!

Tasks

- #1 thing to know about tasks: **tasks are code**, too!

Tasks

- #1 thing to know about tasks: **tasks are code**, too!
 - Should be checked into version control
 - Should be code-reviewed
 - Should be tested

Tasks

- #1 thing to know about tasks: **tasks are code**, too!
 - Should be checked into version control
 - Should be code-reviewed
 - Should be tested
- Tasks also commonly have **dependencies**

Tasks

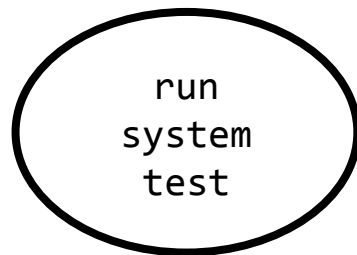
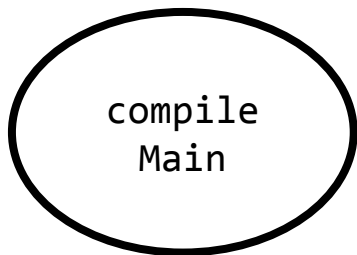
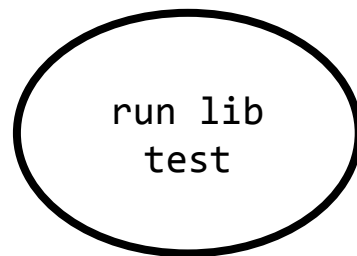
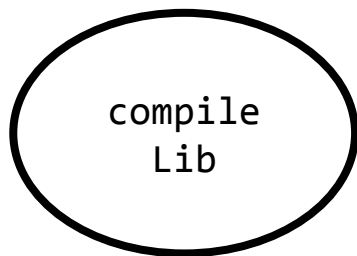
- #1 thing to know about tasks: **tasks are code**, too!
 - Should be checked into version control
 - Should be code-reviewed
 - Should be tested
- Tasks also commonly have **dependencies**
 - Dependency management is a key build system responsibility!

Dependencies between tasks

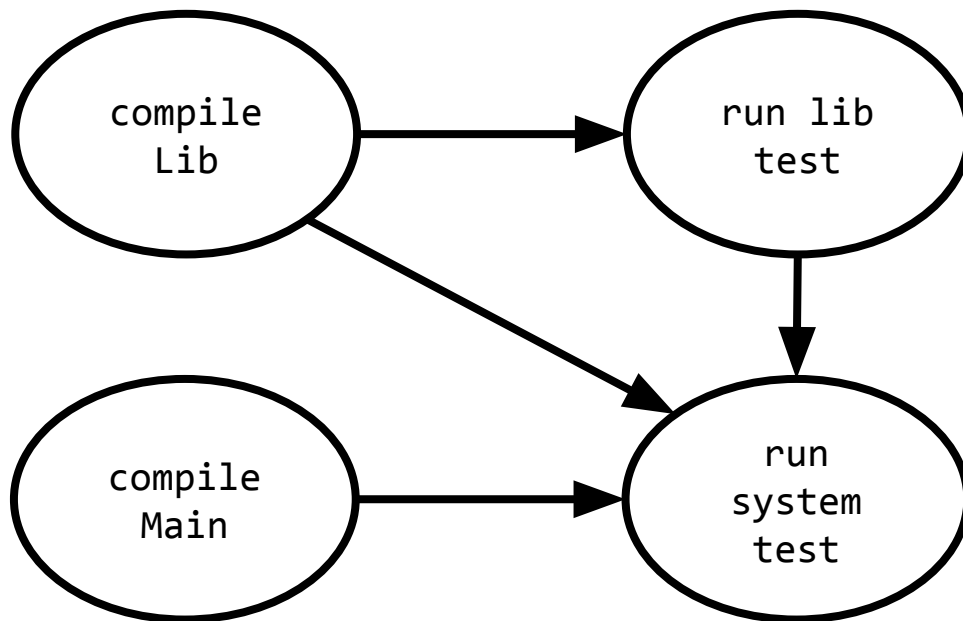
```
> ls src/
```

```
Lib.java    LibTest.java  Main.java    SystemTest.java
```

Dependencies between tasks



Dependencies between tasks



Dependencies between tasks

- A large project may have thousands of tasks

Dependencies between tasks

- A large project may have thousands of tasks
 - What order to run in?
 - How to speed up?

Dependencies between tasks

- A large project may have thousands of tasks
 - **What order to run in?**
 - How to speed up?

Determining task ordering

- Dependencies between tasks form a directed acyclic graph

Determining task ordering

- Dependencies between tasks form a directed acyclic graph

Topological sort!

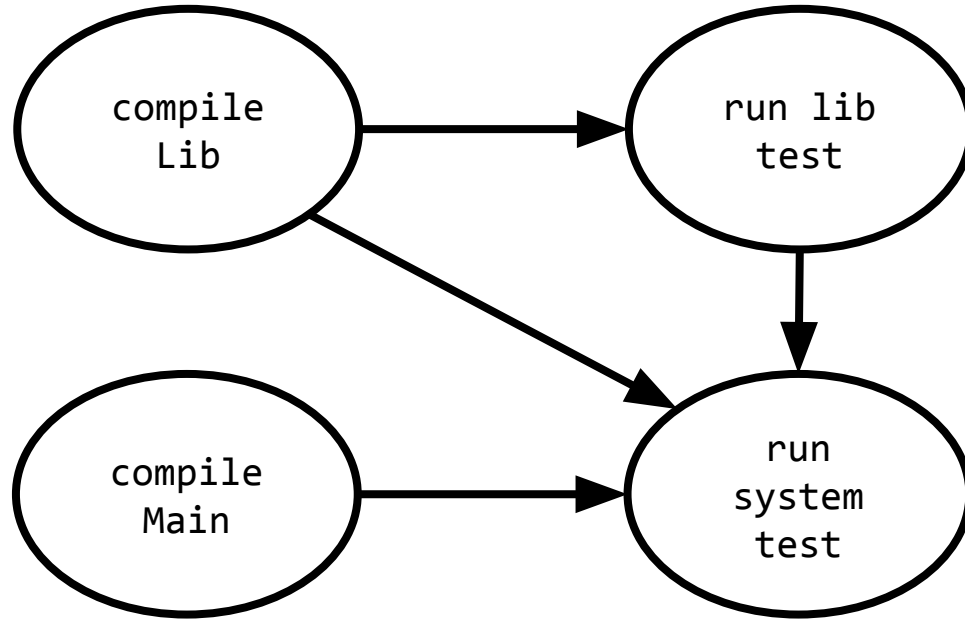
Topological sort

- Any ordering on the nodes such that all dependencies are satisfied

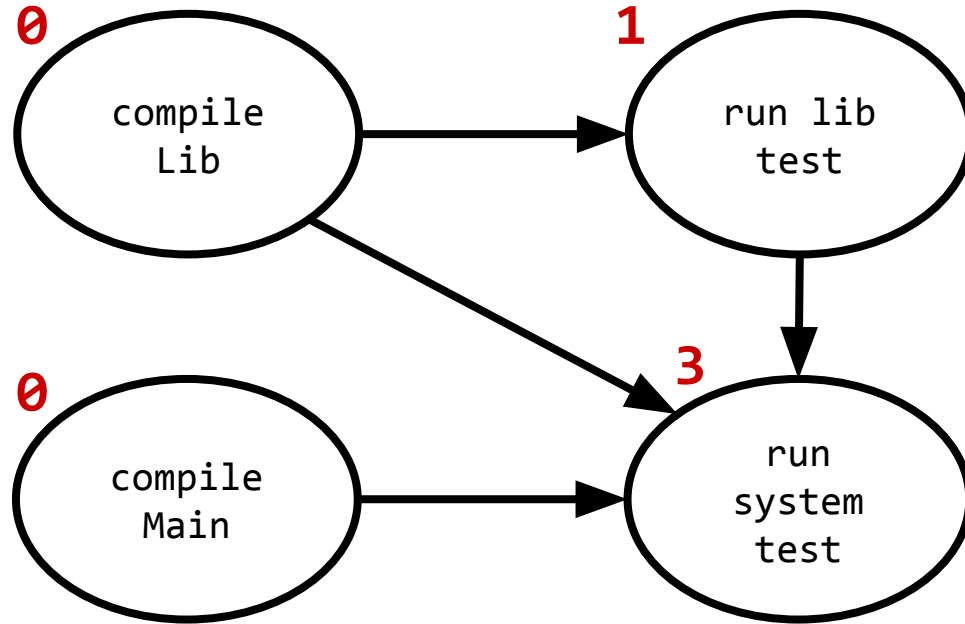
Topological sort

- Any ordering on the nodes such that all dependencies are satisfied
- Implement by computing *indegree* (number of incoming edges) for each node

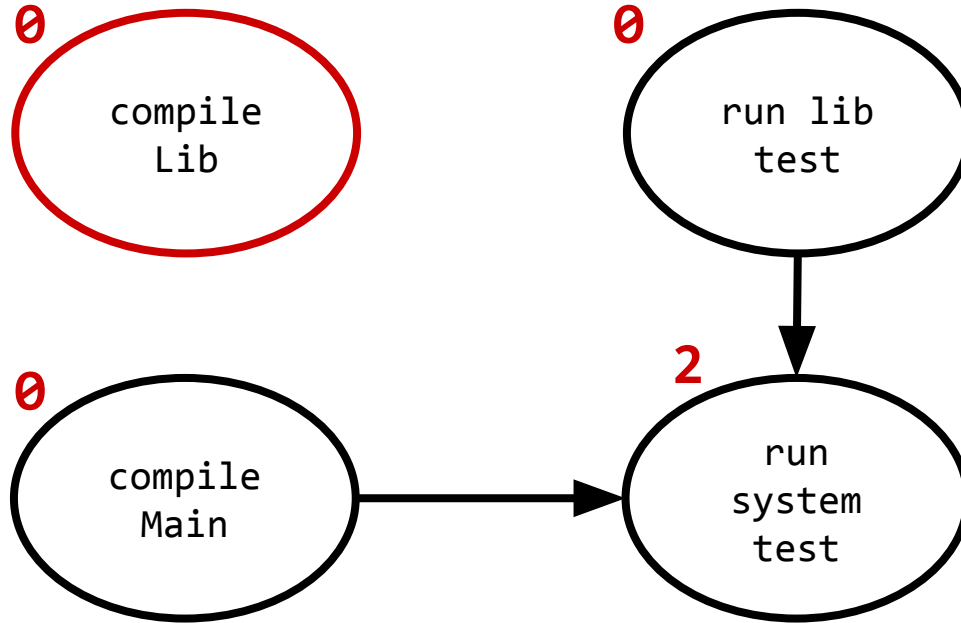
Topological sort



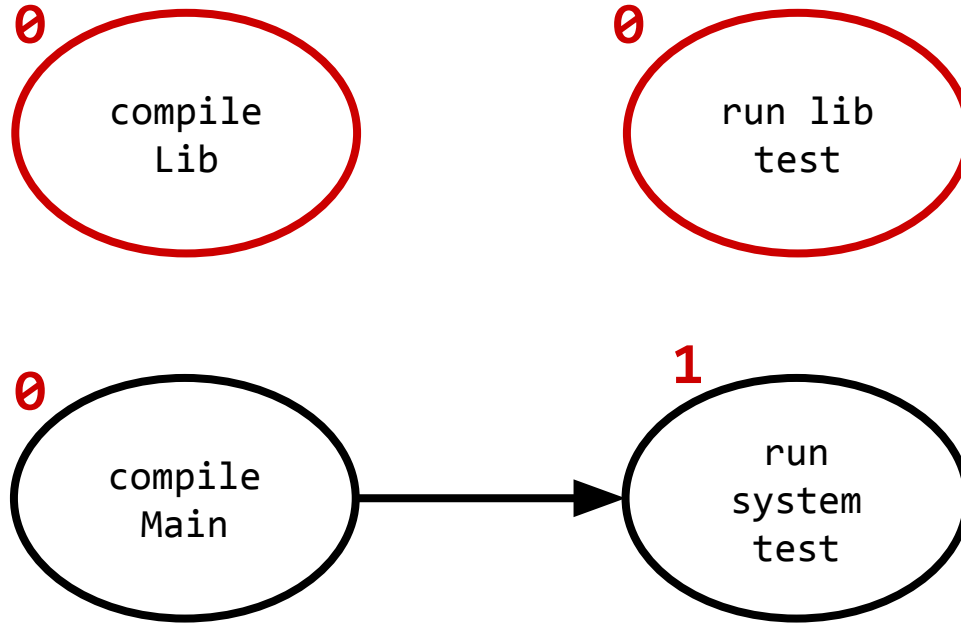
Topological sort



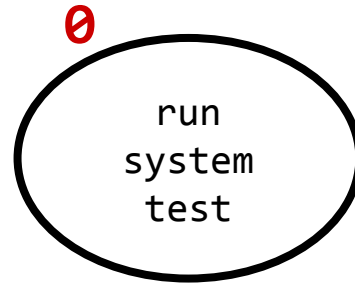
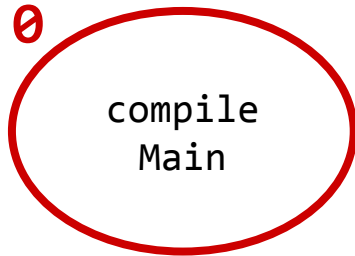
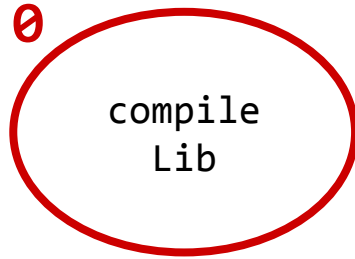
Topological sort



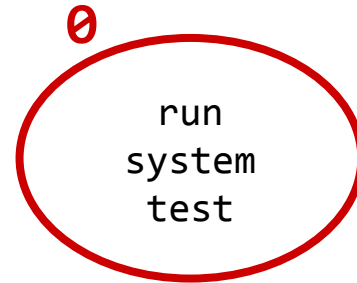
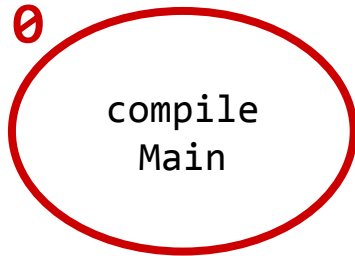
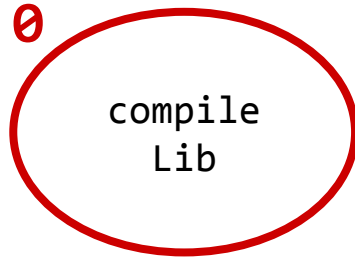
Topological sort



Topological sort



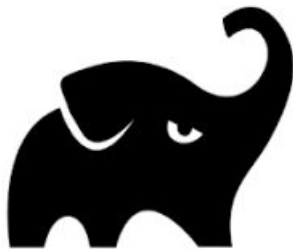
Topological sort



Examples of modern build systems

gradle

<https://gradle.org/>



Apache's open-source successor to ant, maven

bazel

<https://www.bazel.build/>



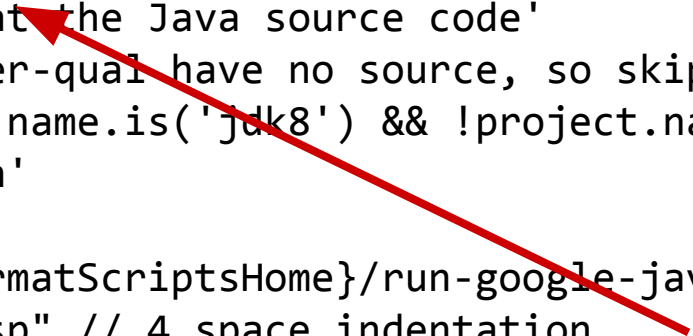
Google's internal build tool, now open-source

Example task: gradle

```
task reformat(type: Exec, dependsOn: getCodeFormatScripts, group: 'Format') {
    description 'Format the Java source code'
    // jdk8 and checker-qual have no source, so skip
    onlyIf { !project.name.is('jdk8') && !project.name.is('checker-qual') }
    executable 'python'
    doFirst {
        args += "${formatScriptsHome}/run-google-java-format.py"
        args += "--aosp" // 4 space indentation
        args += getJavaFilesToFormat(project.name)
    }
}
```

Example task: gradle

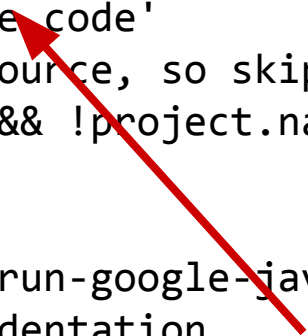
```
task reformat(type: Exec, dependsOn: getCodeFormatScripts, group: 'Format') {  
    description 'Format the Java source code'  
    // jdk8 and checker-qual have no source, so skip  
    onlyIf { !project.name.is('jdk8') && !project.name.is('checker-qual') }  
    executable 'python'  
    doFirst {  
        args += "${formatScriptsHome}/run-google-java-format.py"  
        args += "--aosp" // 4 space indentation  
        args += getJavaFilesToFormat(project.name)  
    }  
}
```



kind of rule

Example task: gradle

```
task reformat(type: Exec, dependsOn: getCodeFormatScripts, group: 'Format') {  
    description 'Format the Java source code'  
    // jdk8 and checker-qual have no source, so skip  
    onlyIf { !project.name.is('jdk8') && !project.name.is('checker-qual') }  
    executable 'python'  
    doFirst {  
        args += "${formatScriptsHome}/run-google-java-format.py"  
        args += "--aosp" // 4 space indentation  
        args += getJavaFilesToFormat(project.name)  
    }  
}
```



explicitly specified dependencies

Example task: gradle

```
task reformat(type: Exec, dependsOn: getCodeFormatScripts, group: 'Format') {  
    description 'Format the Java source code'  
    // jdk8 and checker-qual have no source, so skip  
    onlyIf { !project.name.is('jdk8') && !project.name.is('checker-qual') }  
    executable 'python'  
    doFirst {  
        args += "${formatScriptsHome}/run-google-java-format.py"  
        args += "--aosp" // 4 space indentation  
        args += getJavaFilesToFormat(project.name)  
    }  
}
```



code!

Example task: bazel

```
java_binary(  
    name = "dux",  
    main_class = "org.dux.cli.DuxCLI",  
    deps = ["@google_options//:compile",  
            "@checker_qual//:compile",  
            "@google_cloud_storage//:compile",  
            "@slf4j//:compile",  
            "@logback_classic//:compile"],  
    srcs = glob(["src/org/dux/cli/*.java",  
                 "src/org/dux/backingstore/*.java"),  
)
```

Example task: bazel

```
java_binary(  kind of rule
    name = "dux",
    main_class = "org.dux.cli.DuxCLI",
    deps = ["@google_options//:compile",
            "@checker_qual//:compile",
            "@google_cloud_storage//:compile",
            "@slf4j//:compile",
            "@logback_classic//:compile"],
    srcs = glob(["src/org/dux/cli/*.java",
                 "src/org/dux/backingstore/*.java"),
)
```

Example task: bazel

```
java_binary(  
    name = "dux",  
    main_class = "org.dux.cli.DuxCLI",  
    deps = ["@google_options//:compile",  
            "@checker_qual//:compile",  
            "@google_cloud_storage//:compile",  
            "@slf4j//:compile",  
            "@logback_classic//:compile"],  
    srcs = glob(["src/org/dux/cli/*.java",  
                 "src/org/dux/backingstore/*.java"),  
)
```

explicitly specified
dependencies



Example task: bazel

```
java_binary(  
    name = "dux",  
    main_class = "org.dux.cli.DuxCLI",  
    deps = ["@google_options//:compile",  
            "@checker_qual//:compile",  
            "@google_cloud_storage//:compile",  
            "@slf4j//:compile",  
            "@logback_classic//:compile"],  
    srcs = glob(["src/org/dux/cli/*.java",  
                 "src/org/dux/backingstore/*.java"),  
)
```

explicitly specified
dependencies
(also bazel tasks)

External and internal dependencies

- A list of tasks (internal) or libraries (external)

External and internal dependencies

- A list of tasks (internal) or libraries (external)

```
deps = ["@google_options//:compile",  
        "@checker_qual//:compile",  
        "@google_cloud_storage//:compile",  
        "@slf4j//:compile",  
        "@logback_classic//:compile"],
```

```
dependencies {  
    compile group:  
        'org.hibernate',  
        name: 'hibernate-core',  
        version: '3.6.7.Final'  
    testCompile group:  
        'junit',  
        name: 'junit',  
        version: '4.+'  
}
```

Why list dependencies?

- Reproducibility!

Why list dependencies?

- Reproducibility!
- *Hermetic builds*: “they are insensitive to the libraries and other software installed on the build machine”¹

¹<https://landing.google.com/sre/sre-book/chapters/release-engineering/>

Why list dependencies?

- Reproducibility!
- **Hermetic builds**: “they are insensitive to the libraries and other software installed on the build machine”¹
 - critical if you want to get new developers working quickly (remember the reading!)
 - useful for debugging problems users encounter with old versions (can always get back to exactly the code they’re using)
 - prevents “it works on my machine” syndrome

¹<https://landing.google.com/sre/sre-book/chapters/release-engineering/>

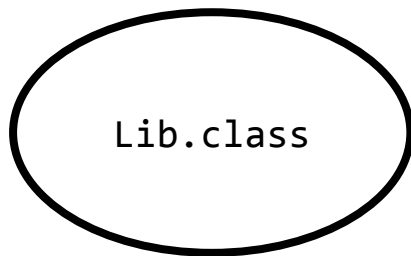
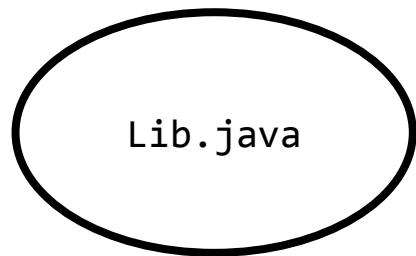
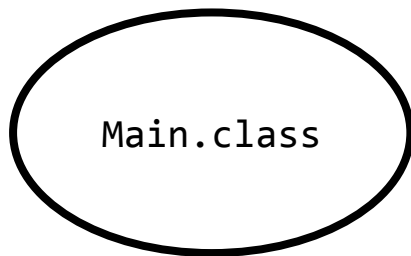
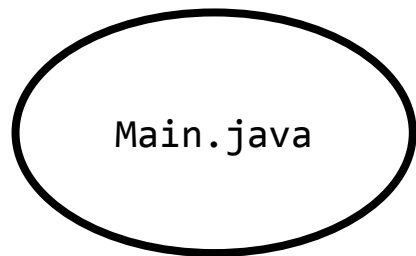
Dependencies between tasks

- A large project may have thousands of tasks
 - What order to run in?
 - **How to speed up?**

How to speed up builds?

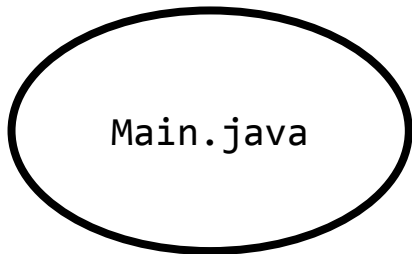
- **Incrementalize** - only rebuild what you have to

Incrementalization

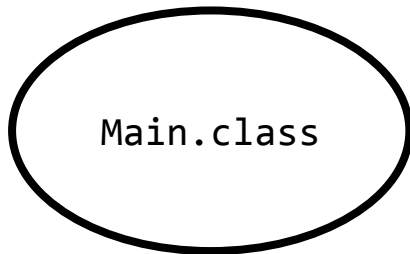


Incrementalization: time stamps

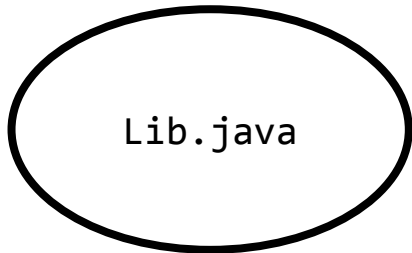
modified 10:45 AM



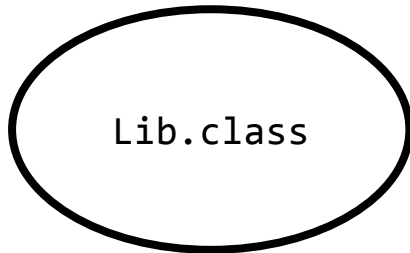
modified 11:06 AM



modified 1:30 PM



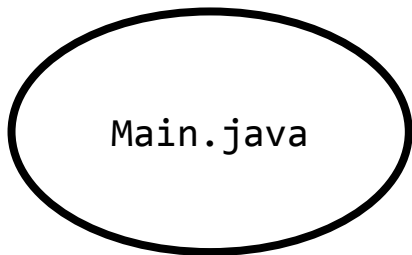
modified 11:06 AM



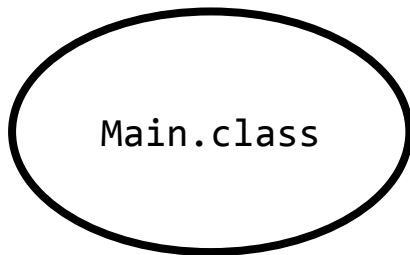
1:31 PM

Incrementalization: time stamps

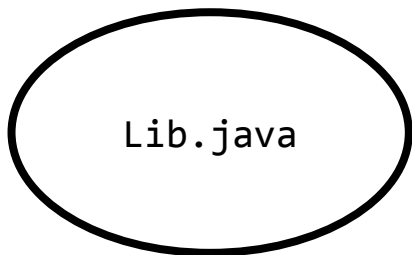
modified 10:45 AM



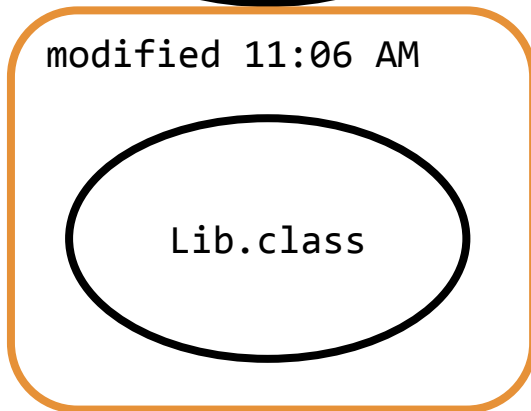
modified 11:06 AM



modified 1:30 PM



modified 11:06 AM

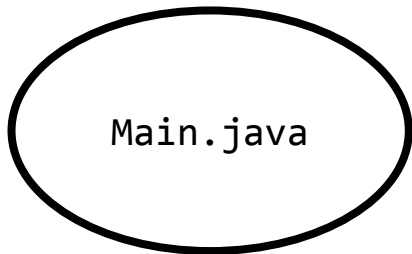


1:31 PM

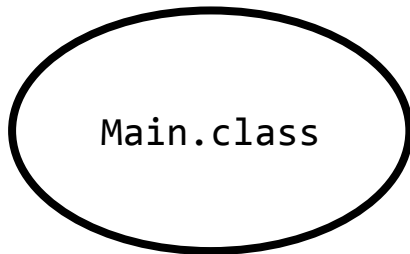
only this file must
be rebuilt

Incrementalization: time stamps

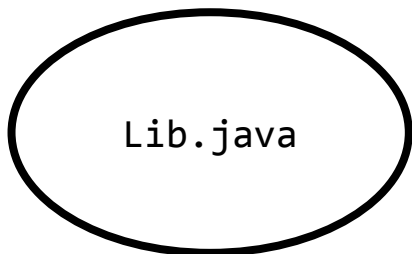
modified 10:45 AM



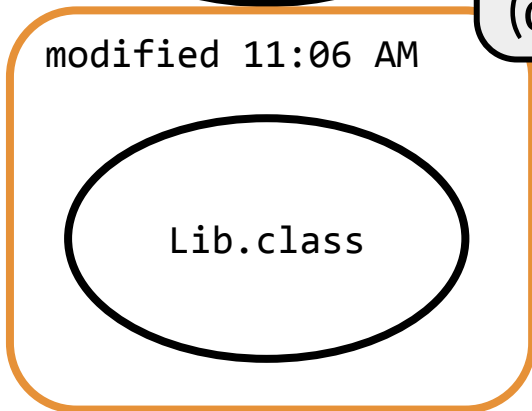
modified 11:06 AM



modified 1:30 PM



modified 11:06 AM



1:31 PM

Subtlety: the clock on a computer can change... (daylight savings, etc.)

only this file must be rebuilt

Incrementalization: hashing

Incrementalization: hashing

- Compute hash codes for inputs to each task
- When about to execute a task, check input hashes - if they match the last time the task was executed, skip it!

How to speed up builds?

- **Incrementalize** - only rebuild what you have to
- Execute many tasks in **parallel**

How to speed up builds?

- **Incrementalize** - only rebuild what you have to
- Execute many tasks in **parallel**
 - some build system tasks are *embarrassingly parallel*: they can be reordered without explicit synchronization
 - is this true of **all** tasks?

How to speed up builds?

- **Incrementalize** - only rebuild what you have to
- Execute many tasks in **parallel**
 - some build system tasks are *embarrassingly parallel*: they can be reordered without explicit synchronization
 - is this true of **all** tasks? **No**: some tasks depend on each other. The problem of scheduling tasks with no unbuilt dependencies is embarrassingly parallel, though.

How to speed up builds?

- **Incrementalize** - only rebuild what you have to
- Execute many tasks in **parallel**
 - some build system tasks are *embarrassingly parallel*: they can be reordered without explicit synchronization
 - is this true of **all** tasks? **No**: some tasks depend on each other. The problem of scheduling tasks with no unbuilt dependencies is embarrassingly parallel, though.
- **Cache** artifacts in the cloud

How do build systems differ

How do build systems differ

- Scheduling algorithm

How do build systems differ

- Scheduling algorithm
 - We've already seen topological scheduling (used by e.g. make), which is a **static** scheduling algorithm

How do build systems differ

- Scheduling algorithm
 - We've already seen topological scheduling (used by e.g. make), which is a **static** scheduling algorithm
 - **Dynamic** scheduling algorithms are also possible

How do build systems differ

- Scheduling algorithm
 - We've already seen topological scheduling (used by e.g. make), which is a **static** scheduling algorithm
 - **Dynamic** scheduling algorithms are also possible
 - **Key idea:** compute what dependencies are necessary as you go

How do build systems differ

- Scheduling algorithm
 - We've already seen topological scheduling (used by e.g. make), which is a **static** scheduling algorithm
 - **Dynamic** scheduling algorithms are also possible
 - **Key idea:** compute what dependencies are necessary as you go
 - this is how e.g., Bazel actually schedules tasks

How do build systems differ

- Rebuilding strategy

How do build systems differ

- Rebuilding strategy
 - We've seen two:

How do build systems differ

- Rebuilding strategy
 - We've seen two:
 - a *dirty bit* strategy (make's timestamps)

How do build systems differ

- Rebuilding strategy
 - We've seen two:
 - a *dirty bit* strategy (make's timestamps)
 - a *verifying trace* strategy (storing hashes of each object)

How do build systems differ

- Rebuilding strategy
 - We've seen two:
 - a **dirty bit** strategy (make's timestamps)
 - a **verifying trace** strategy (storing hashes of each object)
 - Other options:
 - **constructive traces**: store all intermediate objects (usually in the cloud) along with the hashes of the **inputs** used to produce them. If we ever see the same input hashes again, just return the intermediate object

How do build systems differ

- How are tasks expressed?

How do build systems differ

- How are tasks expressed?
 - traditionally **declarative** (e.g., make, Ant, Maven)
 - “declarative” = you tell the build system what you want, it figures out how to build that thing
 - call back to languages: programming languages can also be from the **declarative paradigm** (e.g., Prolog)

How do build systems differ

- How are tasks expressed?
 - traditionally **declarative** (e.g., make, Ant, Maven)
 - “declarative” = you tell the build system what you want, it figures out how to build that thing
 - call back to languages: programming languages can also be from the **declarative paradigm** (e.g., Prolog)
 - most modern build systems have **scripting languages**
 - e.g., Groovy in Gradle, Starlark in Bazel, etc.
 - enables us to write tasks as if they are other code

Best practices

- Automate everything

Best practices

- Automate everything
- Always use a build tool

Best practices

- Automate everything
- Always use a build tool
- Have a build server that builds and tests your code on every commit (continuous integration)

Best practices

- Automate everything
- Always use a build tool
- Have a build server that builds and tests your code on every commit (continuous integration)

Your CI server is a good place to test that your build is hermetic.
Standard practice: spin up a new CI server for **each build**.

Best practices

- Automate everything
- Always use a build tool
- Have a build server that builds and tests your code on every commit (continuous integration)
- Don't depend on anything that's not in the build file (hermetic)

Best practices

- Automate everything
- Always use a build tool
- Have a build server that builds and tests your code on every commit (continuous integration)
- Don't depend on anything that's not in the build file (hermetic)
- Don't break the build

Best practices

- Automate everything
- Always use a build tool
- Have a build server that builds and tests your code on every commit (continuous integration)
- Don't depend on anything that's not in the build file (hermetic)
- Don't break the build

A **common mistake to avoid**: allowing the CI server to fail for a long time because “we know what the problem is.” Don't do this: leads to complacency, missing real bugs.

Takeaways: version control and build system

- Understand what your VCS is good for (storing text files, collaboration) and what it isn't good for (e.g., storing binaries!)
- Use a build system to avoid depending on manual processes
- Follow best practices:
 - don't push straight to main
 - use process to try to avoid merge conflicts, if possible
 - commit early and often, pull as often as you can
 - keep continuous integration green
 - don't allow your build to depend on anything that's not in the build file